AgentForge: Features for an n8n-style AI Agent Studio

A gap analysis based on the graphify knowledge graph of this repository (277 nodes, 465 edges, 16 communities), 11 Nov 2026.

1. What AgentForge already has

The graph shows a working base. These are the parts you build on:

Area	Current implementation
Visual graph editor	React Flow canvas (*GraphEditorCanvas*, *addNode*, *saveGraph*, *updateSelectedConfig*) in apps/web/app/editor/[gid]/graph-editor.tsx.
Graph storage & CRUD	FastAPI routes create/list/get/update/delete graph; *GraphRecord*, *RunRecord*, *ChatMessageRecord* models in Postgres. Supabase mirror in lib/supabase.ts.
Validation	*compile_graph()* checks the graph before it runs and raises *GraphValidationError*. The README requires a DAG reachable from Start to End.
Durable execution	Temporal *RunGraphWorkflow* calls the *execute_node* activity on the *agentforge* task queue; runs start through *start_run()*.
4 node types	**start**, **end**, **template** (Jinja via *render_template*) and **http** (with SSRF guard *validate_public_http_url*).
AI Copilot	*suggest_graph* / *stream_suggest_graph* generate graphs from a prompt (OpenAI-compatible), streamed over SSE to a chat panel (*ChatUI*, *readSSEStream*).
Chat memory & rate limiting	Per-session chat history routes. A Redis sliding-window limiter on the API, and an in-memory limiter on the Next.js chat route.

Key limitation: *RunGraphWorkflow.run()* loops over *payload['nodes']* one after another and passes *last_output* along. The engine ignores edges, so branches, parallel paths, loops and merges cannot work yet. Most of the features below depend on fixing this first.

2. Workflow execution engine (the core)

Feature	What it does	Builds on (from graph)	Priority
DAG scheduler	Runs nodes in topological order using the edges. Nodes with no dependency between them run in parallel, and each node waits until all of its inputs are ready.	RunGraphWorkflow, compile_graph	**P0**
IF / Switch node	Sends data down different branches based on a condition or expression (true/false, or several cases).	compiler, execute_node	**P0**
Merge node	Joins branches back together: append, combine by key, or wait for all inputs.	DAG scheduler	**P0**
Loop / Split-in-batches	Runs part of the graph once per item or batch of an array, then collects the results.	Temporal child workflows	**P1**
Item-based data model	Every node takes in and returns a list of JSON items (plus binary files). This gives all nodes a predictable interface for mapping data.	execute_node signature	**P0**
Expression engine	Lets any parameter refer to earlier nodes, e.g. {{ $node['HTTP'].json.id }}, {{ $input }}, $now. Extends the current Jinja templating.	render_template	**P0**
Error handling	Retries per node with backoff, timeouts, a 'continue on fail' option, an error output branch, and a global error workflow.	Temporal retry policies	**P1**

Feature	What it does	Builds on (from graph)	Priority
Wait / Delay node	Pauses a run for a set time, until a date, or until a webhook resumes it. Temporal timers keep the wait durable.	Temporal timers/signals	P1
Sub-workflow node	Calls another saved workflow as a single step, passing inputs and getting outputs back. This lets workflows be reused.	start_run, child workflows	P1
Partial execution & pinned data	Runs one node, or everything from one node onward, reusing cached or pinned outputs from earlier nodes so you don't have to re-run the whole graph.	RunRecord outputs	P1

3. Triggers (how a workflow starts)

Feature	What it does	Builds on (from graph)	Priority
Manual trigger	A 'Test workflow' button that starts a run with sample input. Partly exists through start_run.	start_run()	P0
Webhook trigger	Gives each workflow its own URL (GET/POST). The incoming request becomes the input, and the workflow can either reply right away or send back the last node's output.	FastAPI router	P0
Schedule / Cron trigger	Runs a workflow on an interval or cron expression, using Temporal Schedules.	Temporal	P0
Chat trigger	A hosted or embeddable chat window where each message starts the agent workflow. Builds on the existing ChatUI and SSE.	ChatUI, messages routes	P1
App event triggers	Starts a workflow on an outside event: new email, a Slack message, a database row, a queue message. Works by polling or by webhooks.	Connector SDK	P2
Activate / deactivate	Turns a workflow's production triggers on or off, separate from test runs.	GraphRecord status field	P0

4. AI agent nodes (what makes it an agent studio)

Feature	What it does	Builds on (from graph)	Priority
LLM / Chat Model node	Calls an LLM (Anthropic Claude, OpenAI, Gemini, Ollama, and so on) with a system prompt, a user prompt, temperature, max tokens, and a model you pick from a list.	copilot.py provider config	P0
AI Agent node	Runs a tool-calling loop: the model decides which tool to call, gets the result, repeats, and finishes with a final answer. Settings for max iterations, system prompt and output format.	execute_node	P0
Tool sub-nodes	Things that plug into an agent as tools: HTTP request, Code, Calculator, Web search, another workflow, and database query. Each has a name, a description and a JSON schema for its inputs.	http node, SSRF guard	P0
MCP client tool	Connects to any Model Context Protocol server and exposes its tools to the agent.	Tool sub-nodes	P1
Memory sub-nodes	Conversation memory per session: full buffer, a window of recent messages, a running summary, or Postgres/Redis storage.	ChatMessageRecord, Redis	P0
Structured output parser	Makes the LLM return JSON that matches a schema. Invalid output is validated and retried automatically.	Pydantic schemas	P1

Feature	What it does	Builds on (from graph)	Priority
RAG pipeline nodes	Nodes for document loaders (PDF, web, Drive), text splitters, embeddings, a vector store (pgvector on the existing Postgres, or Qdrant/Pinecone), and a retriever tool.	postgres:17 service	P1
Multi-agent orchestration	A supervisor agent that hands tasks to specialist agents (sub-workflows) and combines their answers.	Sub-workflow node	P2
Human-in-the-loop	Pauses the agent before risky tool calls or at checkpoints. Someone approves, edits or rejects it from the UI, Slack or email, then the run continues.	Temporal signals	P1
Guardrails	Detects PII, filters prompt injection, moderates content, and limits how many tokens or how much money a run can spend.	LLM node	P2
Prompt library	Saved, versioned prompts with variables that can be reused across nodes.	GraphRecord pattern	P2

5. Integration & data nodes

Feature	What it does	Builds on (from graph)	Priority
Credentials vault	Stores API keys and OAuth tokens encrypted (AES-GCM with a master key). Nodes refer to credentials by ID, and secret values never reach the browser or the logs.	Settings, Postgres	P0
OAuth2 connect flow	A 'Connect account' button for Google, Slack, GitHub and others. Tokens refresh automatically.	Credentials vault	P1
Code node	Runs user JavaScript or Python against the items in a sandbox (separate process or container, with time and memory limits).	worker activities	P0
Data transform nodes	Set/Edit fields, Filter, Sort, Limit, Aggregate, Remove duplicates, Rename keys, Date and time.	Item data model	P0
App connectors	Ready-made nodes for Slack, Gmail, Google Sheets, Notion, Postgres/MySQL, Telegram, Discord, S3, and more.	Node SDK	P1
Binary / file handling	Reads, writes, uploads and converts files (PDF, CSV, images) as items move between nodes.	Item data model	P1
Respond-to-webhook node	Sends a custom HTTP response (status, headers, body) back to the webhook caller.	Webhook trigger	P1

6. Editor & developer experience

Feature	What it does	Builds on (from graph)	Priority
Node palette	A searchable panel of nodes grouped into categories (Triggers, AI, Data, Apps). Drag a node onto the canvas or click + on an edge to add one.	addNode(), nodeTypes	P0
Schema-driven parameter forms	Each node type declares its parameters (type, options, required, shown only when another field has a value). The UI builds the form from that schema instead of hard-coding it.	updateSelectedConfig	P0
Node I/O inspector	Shows each node's input and output for the last run as JSON, a table, or a schema view.	RunRecord	P0
Drag-to-map & autocomplete	Drag a field from an earlier node's output into a parameter to insert the expression. The expression editor autocompletes and previews the result live.	Expression engine	P1

Feature	What it does	Builds on (from graph)	Priority
Live run visualization	Highlights each node as it runs, succeeds or fails, streamed over SSE or WebSocket. Item counts appear on the edges.	readSSEStream	P0
Canvas editing basics	Undo/redo, copy/paste, multi-select, auto-layout, sticky notes, minimap, keyboard shortcuts.	React Flow	P1
Workflow versioning	Saves a version on every save or publish, shows a diff between versions, and can roll back. Draft and published versions are kept separate.	GraphRecord	P1
Import / export & templates	Exports a workflow as JSON and pastes or imports it back. A template gallery of starter agents (support bot, RAG Q&A, lead enrichment).	starterDefinition	P1
Copilot 2.0	The current Copilot generates whole graphs. Version 2 also edits the graph you have open ('add a Slack step after HTTP'), explains a node, and fixes a failed run from its error.	stream_suggest_graph	P2

7. Executions, observability & evaluation

Feature	What it does	Builds on (from graph)	Priority
Execution history	A list of runs with filters (status, trigger, date). Each run opens to show per-node data, timing and errors.	list_runs, get_run	P0
Retry / replay run	Re-runs a failed execution from the node that failed, using the original input.	Temporal history	P1
LLM trace view	For each agent step, shows the prompt, tool calls, model responses, token usage, latency and cost.	AI Agent node	P1
Cost & usage dashboard	Tokens and money spent per workflow, model and user, with budget alerts.	LLM trace data	P2
Evaluations	Runs an agent against a test dataset and scores the answers (exact match, LLM-as-judge), so prompt versions can be compared.	Execution engine	P2
Data retention	Automatically deletes old execution data and masks sensitive fields in logs.	RunRecord	P2

8. Platform, security & scale

Feature	What it does	Builds on (from graph)	Priority
Authentication & users	Sign-up and login (email or SSO), sessions, and API keys for the public API.	Supabase auth	P0
Workspaces & RBAC	Projects or teams with Owner/Editor/Viewer roles. Workflows and credentials can be shared.	Auth	P1
Node SDK / plugin system	A standard way to define nodes (metadata, parameter schema, execute function) and register them, so new or community nodes don't need changes to the core.	execute_node dispatch	P0
Public REST API	Endpoints to create and run workflows and read executions, protected by API keys and rate limits.	FastAPI, RedisRateLimiter	P1
Environment variables	Values per environment (dev/prod) that expressions can use.	Settings	P2
Worker scaling	Several Temporal workers, per-tenant task queues, and limits on concurrent runs.	worker service	P1

Feature	What it does	Builds on (from graph)	Priority
Audit log	Records who changed which workflow or credential, and when.	Postgres	P2

9. Suggested build order

Phase	Scope
Phase 1: Real engine	DAG scheduler, item data model, expression engine, IF and Merge nodes, Node SDK, schema-driven forms, node I/O inspector, live run view.
Phase 2: Agents	Credentials vault, LLM node, AI Agent node, tool sub-nodes, memory sub-nodes, Code node, data transform nodes.
Phase 3: Triggers & production	Webhook and cron triggers, activate/deactivate, execution history, error handling, authentication.
Phase 4: Depth	RAG nodes on pgvector, MCP tools, human-in-the-loop, sub-workflows, loops, versioning, templates, LLM traces.
Phase 5: Scale & polish	App connectors, OAuth, RBAC, evaluations, cost dashboard, multi-agent, guardrails, Copilot 2.0.

10. Issues found in the graph (fix first)

- **Malformed docker-compose:** the *web* service's *depends_on* block in infra/docker-compose.yml appears to contain a pasted redis service definition plus a stray *api:* key (flagged as AMBIGUOUS in the graph).
- **Linear executor:** *RunGraphWorkflow* ignores the graph edges (see section 1).
- **Two API entry points:** *app/main.py* and *main_full.py* both define *health()* and *lifespan()*. Pick one.
- **Two persistence paths:** lib/supabase.ts and the FastAPI/Postgres routes both store graphs and chat history. Pick one source of truth.
- **In-memory rate limiter in Next.js:** lib/rate-limiter.ts resets on every deploy and doesn't work across instances. Use the Redis limiter instead.
- **Low cohesion:** 'Graph Compiler & DB Models' (0.07) and 'Graph Editor Canvas' (0.09) are weakly connected inside. Split graph-editor.tsx into palette, canvas, config panel and inspector components.

Priority key: **P0** needed for an MVP studio · **P1** needed to match n8n · **P2** sets AgentForge apart, or adds polish.